

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ**

Звіт
з лабораторної роботи
**«Рефакторинг коду та проведення Code Review
за індустріальними стандартами»**

Виконав: студент групи ПЗПІ-25-4
Гоков Дмитро Олегович
Перевірив: В. О. Гребенюк

Харків 2026

4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

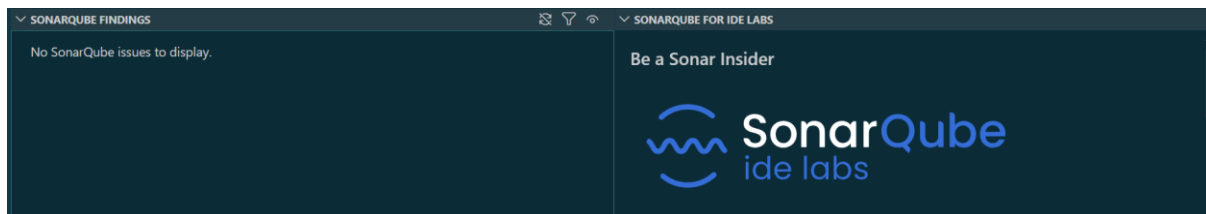
4.2 Хід роботи

4.2.1 Посилання на код модуля та тести з ЛР 3:

<https://github.com/mk1234717/MenuQR/tree/gokov-work/lr3>

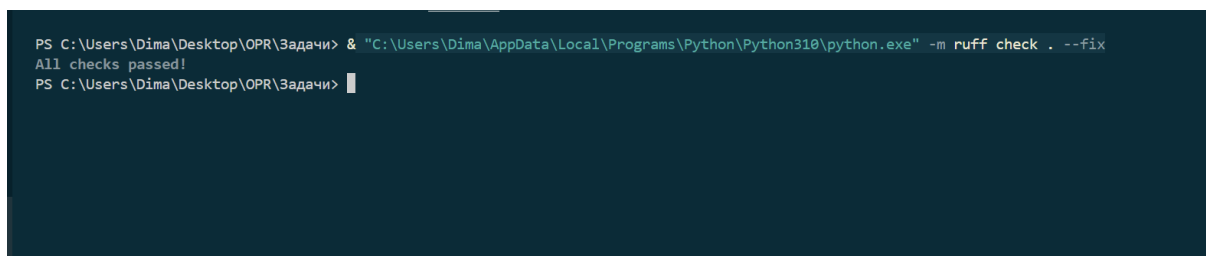
4.2.2 Встановлення SonarQube for IDE (SonarLint) у VS Code:

Початкова кількість проблем = 0



4.2.3 Запуск лінера Ruff:

Початкова кількість попереджень = 0



4.2.4 Обчислення метрик цикломатичної складності та індексу супроводжуваності:

```
PS C:\Users\Dima\Desktop\OPR\Задачи> & "C:\Users\Dima\AppData\Local\Programs\Python\Python310\python.exe" -m radon cc . -a
order.py
  C 12:0 OrderItem - B
  M 14:4 OrderItem.__init__ - B
  M 71:4 Order.calculate_total - B
  C 26:0 Order - A
  M 39:4 Order.add_item - A
  M 28:4 Order.__init__ - A
  M 54:4 Order.apply_promo_code - A
  C 6:0 OrderStatus - A
test_order.py
  F 7:0 test_add_item_success - A
  F 30:0 test_add_item_quantity_boundaries - A
  F 94:0 test_calculate_total_tip_boundaries - A
  F 48:0 test_apply_promo_code_valid - A
  F 70:0 test_calculate_total_empty - A
  F 81:0 test_calculate_total_with_discount_and_tips - A
  F 20:0 test_add_item_wrong_status - A
  F 59:0 test_apply_promo_code_invalid - A
  F 113:0 test_order_item_invalid_name - A
  F 118:0 test_order_invalid_table - A

18 blocks (classes, functions, methods) analyzed.
Average complexity: A (3.111111111111111)
PS C:\Users\Dima\Desktop\OPR\Задачи> & "C:\Users\Dima\AppData\Local\Programs\Python\Python310\python.exe" -m radon mi . -s
order.py - A (51.46)
test_order.py - A (73.00)
PS C:\Users\Dima\Desktop\OPR\Задачи>
```

4.2.5 Проведення Code Review:

№	Рядо к коду	Проблема	Категорі я code smell	Конкретна рекомендація щодо виправлення
1	33– 34	Число 1_000_000 використовується у коді для перевірки максимальної ціни страви без будь-якого пояснення або прив'язки до контексту. Це класичний приклад "магічного числа", який ускладнює розуміння бізнес-логіки та робить код менш гнучким для змін.	Магічне число (Magic Numbers)	Винести це числове значення у чітко іменовану константу на рівні класу.
2	35	У рядку 35 числа 0 та 100 використовуються як межі для перевірки допустимого діапазону знижки без використання іменованих констант.	Магічне число (Magic Numbers)	Винести ці числові літерали у зрозумілі константи на рівні класу.

№	Рядо к коду	Проблема	Категорія code smell	Конкретна рекомендація щодо виправлення
		Це ускладнює розуміння коду та його подальше супроводження, якщо бізнес-правила щодо максимальної чи мінімальної знижки зміняться.		
3	97–100	Метод <code>validate_for_publish</code> містить близько 35 рядків коду і виконує перевірку 4 незалежних полів (назва, ціна, знижка, URL зображення). Це призводить до порушення принципу SRP (Single Responsibility Principle) на рівні методу, ускладнює його читання та підтримку.	Довгий метод (Long Method)	Декомпонувати метод, винісши логіку валідації кожного поля в окремі приватні/захищені методи: <pre> _validate_name(self, errors) _validate_price(self, errors) _validate_discount(self, errors) _validate_image_url(self, errors) </pre>
4	114–115	У рядку 114 виконується перевірка <code>if self.price < 0</code> , яка ніколи не спрацює (є недосяжним кодом). Конструктор класу <code>__init__</code> уже валідує	Мертвий код (Dead Code)	Видалити рядки 114–115 як мертвий код для очищення логіки методу.

№	Рядок коду	Проблема	Категорія code smell	Конкретна рекомендація щодо виправлення
		вхідні дані й викидає ValueError у разі від'ємної ціни страви, тому створення об'єкта з таким станом є неможливим.		
5	127–130	Інструкція import re знаходиться всередині методу validate_for_publish. Через це модуль імпортується при кожному виклику функції, що навантажує систему, ускладнює читання коду та порушує стандарт PEP 8.	Поганий стиль (Poor Style)	Перенести import re на початок файлу (рядок 1) разом з іншими імпортами.

Зауваження було задокументовано у вигляді inline-коментарів у PR:

<https://github.com/mk1234717/MenuQR/pull/8>

4.2.6 Рефакторинг коду Було отримано результати Code Review:

<https://github.com/mk1234717/MenuQR/pull/7>

№1. Заміна магічного числа константою:

Проблема: Межа кількості однієї страви (20) використана безпосередньо у коді.

БУЛО:

```
def __init__(self, name: str, price: float, quantity: int):
    if not name or not name.strip():
        raise ValueError("Назва страви не може бути порожньою.")
    if price <= 0:
        raise ValueError("Ціна страви повинна бути більшою за 0.")
    if quantity <= 0 or quantity > 20:
        raise ValueError("Кількість однієї страви повинна бути в межах від 1 до 20.")
```

СТАЛО:

```
class OrderItem:
    MAX_QUANTITY_PER_ITEM = 20 # константа класу

    def __init__(self, name: str, price: float, quantity: int):
        # ...
        if quantity <= 0 or quantity > self.MAX_QUANTITY_PER_ITEM:
            raise ValueError(f"Кількість однієї страви повинна бути в межах
від 1 до {self.MAX_QUANTITY_PER_ITEM}.")
```

ЧОМУ:

Усунено дублювання (DRY): Число 20 тепер визначене лише в одному місці (OrderItem.MAX_QUANTITY_PER_ITEM). Метод add_item у класі Order посилається на нього, що гарантує синхронність лімітів у всій програмі.

```
rootdir: C:\Users\Dima\Desktop\OPR\Задачи
plugins: cov-7.1.0
collected 10 items

test_order.py::test_add_item_success PASSED [ 10%]
test_order.py::test_add_item_wrong_status PASSED [ 20%]
test_order.py::test_add_item_quantity_boundaries PASSED [ 30%]
test_order.py::test_apply_promo_code_valid PASSED [ 40%]
test_order.py::test_apply_promo_code_invalid PASSED [ 50%]
test_order.py::test_calculate_total_empty PASSED [ 60%]
test_order.py::test_calculate_total_with_discount_and_tips PASSED [ 70%]
test_order.py::test_calculate_total_tip_boundaries PASSED [ 80%]
test_order.py::test_order_item_invalid_name PASSED [ 90%]
test_order.py::test_order_invalid_table PASSED [100%]

===== 10 passed in 0.07s =====
PS C:\Users\Dima\Desktop\OPR\Задачи> |
```

№2. Заміна магічного числа константою:

Проблема: Межі номера столика (1 та 50) задані як літерали.

БУЛО:

```
class Order:
    """Клас замовлення цифрового QR-меню."""
    def __init__(self, table_number: int):
        if table_number <= 0 or table_number > 50:
            raise ValueError("Номер столика повинен бути в межах від 1 до
50.")
```

СТАЛО:

```
class Order:
    """Клас замовлення цифрового QR-меню."""

    # Константи бізнес-логіки
    MAX_TABLE_NUMBER = 50
```

```
def __init__(self, table_number: int):
    if table_number <= 0 or table_number > self.MAX_TABLE_NUMBER:
        raise ValueError(f"Номер столика повинен бути в межах від 1 до {self.MAX_TABLE_NUMBER}.")
```

ЧОМУ:

Усунення «магічних чисел» (Magic Numbers): Граничне значення кількості столиків винесене в константу класу MAX_TABLE_NUMBER. Якщо конфігурація закладу зміниться, ліміт правиться в одному місці.

```
rootdir: C:\Users\Dima\Desktop\OPR\Задачи
plugins: cov-7.1.0
collected 10 items

test_order.py::test_add_item_success PASSED [ 10%]
test_order.py::test_add_item_wrong_status PASSED [ 20%]
test_order.py::test_add_item_quantity_boundaries PASSED [ 30%]
test_order.py::test_apply_promo_code_valid PASSED [ 40%]
test_order.py::test_apply_promo_code_invalid PASSED [ 50%]
test_order.py::test_calculate_total_empty PASSED [ 60%]
test_order.py::test_calculate_total_with_discount_and_tips PASSED [ 70%]
test_order.py::test_calculate_total_tip_boundaries PASSED [ 80%]
test_order.py::test_order_item_invalid_name PASSED [ 90%]
test_order.py::test_order_invalid_table PASSED [100%]

===== 10 passed in 0.48s =====
PS C:\Users\Dima\Desktop\OPR\Задачи>
```

№3. Усунення дублювання ліміту кількості страв:

Проблема: Перевірка кількості страви (не більше 20) повторюється в OrderItem.__init__ (рядок 19) та тут. Зміна ліміту потребує правки у двох місцях.

БУЛО:

```
for item in self.items:
    if item.name == name.strip():
        new_qty = item.quantity + quantity
        if new_qty > 20:
            raise ValueError("Сумарна кількість однієї страви не може перевищувати 20.")
```

СТАЛО:

```
for item in self.items:
    if item.name == name.strip():
        new_qty = item.quantity + quantity
        # Використовуємо константу з класу OrderItem для уникнення
        дублювання

        if new_qty > OrderItem.MAX_QUANTITY_PER_ITEM:
            raise ValueError(f"Сумарна кількість однієї страви не може перевищувати {OrderItem.MAX_QUANTITY_PER_ITEM}.")
```

ЧОМУ:

Дотримання принципу DRY: Логіка обмеження кількості тепер керується з одного джерела правди — глобальної константи класу `OrderItem.MAX_QUANTITY_PER_ITEM`.

```
rootdir: C:\Users\Dima\Desktop\OPR\Задачи
plugins: cov-7.1.0
collected 10 items

test_order.py::test_add_item_success PASSED [ 10%]
test_order.py::test_add_item_wrong_status PASSED [ 20%]
test_order.py::test_add_item_quantity_boundaries PASSED [ 30%]
test_order.py::test_apply_promo_code_valid PASSED [ 40%]
test_order.py::test_apply_promo_code_invalid PASSED [ 50%]
test_order.py::test_calculate_total_empty PASSED [ 60%]
test_order.py::test_calculate_total_with_discount_and_tips PASSED [ 70%]
test_order.py::test_calculate_total_tip_boundaries PASSED [ 80%]
test_order.py::test_order_item_invalid_name PASSED [ 90%]
test_order.py::test_order_invalid_table PASSED [100%]

===== 10 passed in 0.22s =====
```

№4. Усунення магічного числа для ліміту чайових:

Проблема: Максимальний відсоток чайових (50) заданий як літерал.

БУЛО:

```
def calculate_total(self, tip_percent: float = 0.0) -> float:
    if tip_percent < 0 or tip_percent > 50:
        raise ValueError("Чайові повинні бути в діапазоні від 0% до 50%.")
```

СТАЛО:

```
MAX_TIP_PERCENT = 50.0

def calculate_total(self, tip_percent: float = 0.0) -> float:
    if tip_percent < 0 or tip_percent > self.MAX_TIP_PERCENT:
        raise ValueError(f"Чайові повинні бути в діапазоні від 0% до {self.MAX_TIP_PERCENT}%.")
```

ЧОМУ:

Централізація конфігурації: Ліміт винесено в константу класу `MAX_TIP_PERCENT`. Якщо менеджмент ресторану вирішить змінити максимальні чайові, це робиться в одному рядку.


```

test_order.py::test_add_item_success PASSED [ 10%]
test_order.py::test_add_item_wrong_status PASSED [ 20%]
test_order.py::test_add_item_quantity_boundaries PASSED [ 30%]
test_order.py::test_apply_promo_code_valid PASSED [ 40%]
test_order.py::test_apply_promo_code_invalid PASSED [ 50%]
test_order.py::test_calculate_total_empty PASSED [ 60%]
test_order.py::test_calculate_total_with_discount_and_tips PASSED [ 70%]
test_order.py::test_calculate_total_tip_boundaries PASSED [ 80%]
test_order.py::test_order_item_invalid_name PASSED [ 90%]
test_order.py::test_order_invalid_table PASSED [100%]

===== 10 passed in 0.23s =====
PS C:\Users\Dima\Desktop\OPR\Задачи>

```

№5 Усунення втрати точності при приведенні типів (Decimal до float):

Проблема: Розрахунки ведуться в Decimal, але результат перетворюється у float. Це нівелює переваги точного десяткового типу і може спричинити помилки округлення.

БУЛО:

```

if not self.items:
    return 0.0

subtotal = sum(item.price * item.quantity for item in self.items)

```

СТАЛО:

```

if not self.items:
    return Decimal('0.00') # Повертаємо точне значення Decimal

subtotal = sum(item.price * item.quantity for item in self.items)

if self.discount_percent > 0:
    subtotal -= subtotal * (self.discount_percent / Decimal('100'))

tips = subtotal * (Decimal(str(tip_percent)) / Decimal('100'))
total = subtotal + tips

# Повертаємо чистий Decimal із правильним математичним округленням
return total.quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)

```

ЧОМУ:

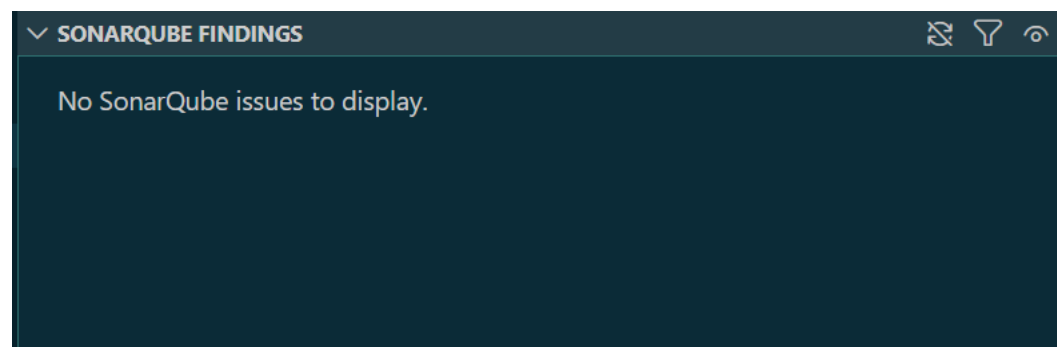
Збереження фінансової точності: Зміна типу повертаного значення на Decimal гарантує, що грошові суми (наприклад, 150.30) не перетворюються на процесорні

артефакти на кшталт 150.30000000000001 через особливості двійчної системи float.

```
collected 10 items
test_order.py::test_add_item_success PASSED [ 10%]
test_order.py::test_add_item_wrong_status PASSED [ 20%]
test_order.py::test_add_item_quantity_boundaries PASSED [ 30%]
test_order.py::test_apply_promo_code_valid PASSED [ 40%]
test_order.py::test_apply_promo_code_invalid PASSED [ 50%]
test_order.py::test_calculate_total_empty PASSED [ 60%]
test_order.py::test_calculate_total_with_discount_and_tips PASSED [ 70%]
test_order.py::test_calculate_total_tip_boundaries PASSED [ 80%]
test_order.py::test_order_item_invalid_name PASSED [ 90%]
test_order.py::test_order_invalid_table PASSED [100%]
===== 10 passed in 0.23s =====
```

4.2.7 Повторний запуск SonarLint та літера. Фіксація кількості issues «після»

Кількість проблем SonarLint = 0



Кількість попереджень Ruff = 0

```
PS C:\Users\Dima\Desktop\OPR\Задачи> & "C:\Users\Dima\AppData\Local\Programs\Python\Python310\python.exe" -m ruff check . --fix
All checks passed!
PS C:\Users\Dima\Desktop\OPR\Задачи>
```

4.2.8 Повторне обчислення метрик цикломатичної складності та індексу супроводжуваності

```
order.py
C 13:0 OrderItem - B
M 83:4 Order.calculate_total - B
C 32:0 Order - A
M 50:4 Order.add_item - A
M 39:4 Order.__init__ - A
M 66:4 Order.apply_promo_code - A
C 6:0 OrderStatus - A
test_order.py
F 7:0 test_add_item_success - A
F 30:0 test_add_item_quantity_boundaries - A
F 94:0 test_calculate_total_tip_boundaries - A
F 48:0 test_apply_promo_code_valid - A
F 70:0 test_calculate_total_empty - A
F 81:0 test_calculate_total_with_discount_and_tips - A
F 20:0 test_add_item_wrong_status - A
F 59:0 test_apply_promo_code_invalid - A
F 113:0 test_order_item_invalid_name - A
F 118:0 test_order_invalid_table - A

18 blocks (classes, functions, methods) analyzed.
Average complexity: A (3.111111111111111)
PS C:\Users\Dima\Desktop\OPR\Задачи> & "C:\Users\Dima\AppData\Local\Programs\Python\Python310\python.exe" -m radon mi . -s
order.py - A (55.36)
test_order.py - A (73.00)
PS C:\Users\Dima\Desktop\OPR\Задачи> 
```

Відрефакторингований код було оновлено та запущено на гітхаб:

<https://github.com/mk1234717/MenuQR/tree/gokov-work/lr%204>